অধ্যায় ৮

Strings এবং Modern Text Processing

শেখার লক্ষ্য

এই অধ্যায় শেষে পাঠক String Primitive, Indexing, Unicode, Normalization, Template Literals, Searching, Splitting, Regular Expression, Safe DOM Rendering এবং Content Analysis বুঝবেন। Practical Project-এ বাংলা ও English কনটেন্ট থেকে Word Count, Slug, Excerpt, Frequency এবং Search Preview তৈরি করবেন।

৮.১ String কী এবং কেন গুরুত্বপূর্ণ?

ওয়েব অ্যাপ্লিকেশনে নাম, ইমেইল, URL, Search Query, Blog Post, API Message—সবই Text Data। JavaScript-এর `string` হলো Primitive Data Type; String Value অপরিবর্তনীয় (*immutable*)। Variable-এ নতুন String Assign করা যায়, কিন্তু আগের String-এর Character সরাসরি পরিবর্তন করা যায় না।

```javascript
const language = 'JavaScript';
console.log(typeof language); // string
console.log(language[0]);     // J
language[0] = 'j';            // Primitive string অপরিবর্তিত থাকে (strict mode-এ error হতে পারে)
console.log(language);        // JavaScript
```

String Object Wrapper ও Primitive String আলাদা। সাধারণ কাজে `new String('hello')` ব্যবহার না করাই ভালো, কারণ সেটি Object এবং Truthiness/Equality-তে বিভ্রান্তি তৈরি করে।

```javascript
const a = 'hello';
const b = new String('hello');
console.log(a === b); // false
console.log(Boolean(new String(''))); // true: Object truthy
```

৮.২ String লেখার তিন পদ্ধতি

Single Quote, Double Quote ও Backtick—সবই বৈধ। Backtick-এর Template Literal Expression Interpolation এবং Multiline Content-এর জন্য উপযোগী।

```javascript
const name = 'Amina';
const city = "Dhaka";
const message = `Hello, ${name}! Welcome to ${city}.`;
const invoice = `Customer: ${name}
Location: ${city}`;
console.log(message);
```

HTML-এ Template Literal দিয়ে User Input সরাসরি `innerHTML`-এ ঢোকানো ঝুঁকিপূর্ণ। Display Text-এর জন্য `textContent` ব্যবহার করুন।

৮.৩ Length, Index ও UTF-16-এর বাস্তবতা

JavaScript String-এর `length` এবং Numeric Index মূলত UTF-16 **Code Unit** গণনা করে, মানুষের চোখে দেখা Character নয়। Emoji ও Combined Character-এ এই পার্থক্য গুরুত্বপূর্ণ।

```javascript
console.log('ABC'.length);      // 3
console.log('🙂'.length);       // 2 (UTF-16 code units)
console.log([...'🙂'].length);   // 1 (Unicode code point)
console.log('👩‍👧'.length);  // 1 নয়; একাধিক code point ও joiner আছে
```

একটি দৃশ্যমান Character (*grapheme cluster*) গণনার জন্য উপযুক্ত Environment-এ `Intl.Segmenter` ব্যবহার করুন। এটি বাংলা যুক্তাক্ষর, Emoji Sequence এবং Combining Mark-এর ক্ষেত্রে সাধারণ `.length`-এর চেয়ে অর্থপূর্ণ ফল দিতে পারে। তবে Segmentation Locale ও Unicode Version অনুযায়ী আচরণ পরিবর্তিত হতে পারে।

```javascript
const segmenter = new Intl.Segmenter('bn', { granularity: 'grapheme' });
const visible = [...segmenter.segment('বাংলা 🙂')].length;
console.log(visible);
```

৮.৪ String Access: `at()`, `charAt()`, `slice()`

```javascript
const text = 'JavaScript';
console.log(text.at(0));      // J
console.log(text.at(-1));     // t
console.log(text.charAt(4));  // S
console.log(text.slice(0, 4)); // Java
console.log(text.slice(-6));  // Script
```

`slice(start, end)`-এর `end` অন্তর্ভুক্ত নয়। `substring()` Negative Index-কে 0 হিসেবে ধরে এবং Start/End উল্টো হলে অদলবদল করতে পারে। নতুন Code-এ `slice()` সাধারণত সহজ ও Predictable। UTF-16 Boundary মাঝখানে কাটলে Emoji ভেঙে যেতে পারে; Unicode-Aware Truncation-এর জন্য Code Point বা Grapheme ভিত্তিক পদ্ধতি দরকার।

৮.৫ Searching: `includes()`, `startsWith()`, `endsWith()`, `indexOf()`

```javascript
const title = 'Learn Modern JavaScript';
console.log(title.includes('Modern'));      // true
console.log(title.startsWith('Learn'));     // true
console.log(title.endsWith('Script'));      // true
console.log(title.indexOf('Java'));         // 13
console.log(title.indexOf('Python'));       // -1
```

Search Case-Sensitive। Case-Insensitive Search-এর সাধারণ কৌশল হলো দুই দিক একই Case-এ রূপান্তর করা। কিন্তু Unicode Case Mapping-এ কিছু ভাষায় String Length পরিবর্তিত হতে পারে; তাই Search Highlight-এর Index Mapping জটিল হয়ে যায়। Production Search-এ Locale-Aware Rules, Tokenization এবং প্রয়োজনে Dedicated Search Engine বিবেচনা করুন।

৮.৬ Clean Text: `trim()`, `replace()`, `replaceAll()`

```javascript
const raw = '   Learn   JavaScript   ';
console.log(raw.trim());                    // Learn   JavaScript
console.log(raw.trim().replace(/\s+/g, ' ')); // Learn JavaScript
console.log('red blue red'.replace('red', 'green')); // green blue red
console.log('red blue red'.replaceAll('red', 'green')); // green blue green
```

`trim()` কেবল String-এর শুরু ও শেষের Whitespace সরায়। সব Internal Whitespace পরিবর্তন করা Content Formatting-এর জন্য সবসময় উপযুক্ত নয়—Poetry, Code Snippet ও Address-এর Line Break হারাতে পারে।

৮.৭ Case Conversion এবং Locale

```javascript
console.log('HELLO'.toLowerCase());
console.log('hello'.toUpperCase());
console.log('İSTANBUL'.toLocaleLowerCase('tr'));
```

Case Folding সব ভাষায় একই নয়। Bangla Script-এ সাধারণত Uppercase/Lowercase নেই। Multilingual Search-এ Locale সম্পর্কে সচেতন থাকুন।

৮.৮ Split, Join ও Tokenization

```javascript
const csvLike = 'apple,banana,mango';
console.log(csvLike.split(','));
console.log(['Learn', 'JavaScript'].join(' '));
const tokens = 'বাংলা JavaScript 2026'.match(/[\p{L}\p{M}\p{N}]+/gu) ?? [];
console.log(tokens);
```

`split(',')` একটি Full CSV Parser নয়: Quoted Field-এর ভেতরে Comma থাকলে ভুল ফল দিতে পারে। একইভাবে Word Tokenization-এর একটি Regular Expression সব ভাষা, Apostrophe, Hyphen, Emoji ও Punctuation নিখুঁতভাবে সামলাতে পারে না। Project-এর Word Count একটি শিক্ষামূলক Heuristic।

৮.৯ Unicode Normalization: NFC বনাম NFD

একই দৃশ্যমান Text Unicode-এ একাধিক Code Point Sequence দিয়ে প্রকাশিত হতে পারে। Normalization Search, Comparison ও Deduplication-এর কাজে সহায়ক।

```javascript
const composed = 'café';
const decomposed = 'cafe\u0301';
console.log(composed === decomposed); // false
console.log(composed.normalize('NFC') === decomposed.normalize('NFC')); // true
```

`NFC` সাধারণ Canonical Composition; `NFD` Decomposition। `NFKC` / `NFKD` Compatibility Transformation করে, যা কিছু প্রতীকের অর্থপূর্ণ পার্থক্যও বদলাতে পারে। Identifier, Legal Text বা Security-Sensitive Data-তে Normalization Policy স্পষ্ট করুন।

৮.১০ Regular Expression-এর পরিচিতি

Regular Expression Pattern Matching-এর শক্তিশালী উপায়। পরবর্তী Advanced Validation অধ্যায়ে আরও গভীরভাবে আলোচনা হবে।

```javascript
const text = 'Order A-102, Order B-205';
console.log(text.match(/[A-Z]-\d{3}/g)); // ['A-102', 'B-205']
console.log(/^\d{4}$/.test('2026'));     // true
```

Regex Flag `g` মানে Global Search, `i` Case-Insensitive, `u` Unicode-Aware Pattern। User-Provided Regex সরাসরি চালালে Invalid Pattern বা ReDoS ঝুঁকি থাকতে পারে। আমাদের Project-এ User Search Query-কে Literal Text হিসেবে Search করা হয়েছে।

৮.১১ Content SEO: Slug, Excerpt, Keyword Frequency

Slug হলো URL-Friendly Path Segment, যেমন `learn-javascript-basics`। English-Only Slug বানাতে ASCII Transliteration প্রচলিত হলেও বাংলা Unicode Slug-ও URL Encoding-এর মাধ্যমে ব্যবহারযোগ্য। Slug Uniqueness, Reserved Words, Maximum Length ও Redirect Strategy Production CMS-এ আলাদা সমস্যা।

```javascript
function slugify(value) {
  return String(value).normalize('NFC').trim().toLocaleLowerCase('en')
    .replace(/[^\p{L}\p{M}\p{N}]+/gu, '-')
    .replace(/^-+|-+$/g, '');
}
console.log(slugify('Learn JavaScript — Step by Step!'));
// learn-javascript-step-by-step
console.log(slugify('বাংলা কনটেন্ট')); // বাংলা-কনটেন্ট
```

Excerpt সংক্ষিপ্ত Preview; এটি HTML Meta Description-এর সঙ্গে সম্পর্কিত হতে পারে, কিন্তু নির্দিষ্ট Character Count থাকলেই SEO Ranking নিশ্চিত হয় না। **Keyword Frequency** শুধু Count; Content Quality বা Search Intent-এর বিকল্প নয়।

৮.১২ Text Security: XSS ও Safe Rendering

Search Highlight দেখানোর সময় একটি সাধারণ ভুল হলো `innerHTML` দিয়ে User Content জুড়ে দেওয়া। এতে HTML Injection/XSS-এর ঝুঁকি তৈরি হয়। নিরাপদ পদ্ধতিতে `document.createTextNode()` এবং `mark.textContent` ব্যবহার করা যায়।

```javascript
const mark = document.createElement('mark');
mark.textContent = '<script>alert(1)</script>'; // Text হিসেবে দেখাবে, চালাবে না
preview.append(mark);
```

`textContent` HTML Interpret করে না। তবে নিরাপত্তা Context-Specific: URL, CSS, Attribute, SQL ও Shell-এর জন্য আলাদা Safe Handling প্রয়োজন।

৮.১৩ Practical Project — Search-Friendly Content Tool

Scenario: একজন Content Editor বাংলা ও English Article লিখছেন। তিনি Word Count, Unicode Character Count, Slug, Short Excerpt, Most Frequent Words এবং Search Match দেখতে চান। আমরা একটি Offline Browser Tool বানাব।

Feature Requirements

1. বাংলা/English Input ও Search Phrase।
2. Word Count, Code Point Count এবং Grapheme Count।
3. Unicode-Compatible Slug Generation।
4. Configurable Excerpt Length।
5. Top Eight Word Frequency।
6. Case-Insensitive Search Highlight—HTML Injection ছাড়া।
7. Responsive UI এবং Clipboard Copy।
8. Content Browser-এ Process হবে; কোনো Server Request নেই।

Project Folder

```
source-code/chapter-08/
├── index.html
├── style.css
├── text-tools.js
├── app.js
└── text-tools.test.js
```

সম্পূর্ণ Runnable Source বইয়ের Companion Source Code-এর `source-code/chapter-08/`-এ সংযুক্ত। নিচে মূল Algorithm এবং Integration ব্যাখ্যা করা হলো।

Step 1: Pure Functions

JAVASCRIPT

```javascript
const normalize = value => String(value ?? '').normalize('NFC').trim();
const words = value => normalize(value).match(/[\p{L}\p{M}\p{N}]+(?:['’][\p{L}\p{M}\p{N}]+)*/gu)
?? [];
const wordCount = value => words(value).length;
const characterCount = value => Array.from(normalize(value)).length;
```

এখানে `String(value ?? '')` Nullish Value-এর জন্য Empty String দেয়। `Array.from` Unicode Code Point Count করে; Grapheme Count নয়। Full Source-এ `Intl.Segmenter` ব্যবহার করা হয়েছে।

Step 2: Frequency Analysis

JAVASCRIPT

```javascript
function frequency(value) {
  const map = new Map();
  for (const word of words(value)) {
    const key = word.toLocaleLowerCase();
    map.set(key, (map.get(key) ?? 0) + 1);
  }
  return [...map].sort((a, b) => b[1] - a[1] || a[0].localeCompare(b[0]));
}
```

`Map` প্রতিটি Word-এর Count ধরে। Sort Frequency অনুযায়ী Descending, সমান হলে Alphabetic। N হলো Token Count, U হলো Unique Word Count: Counting প্রায় O(N), Sorting প্রায় O(U log U)।

Step 3: Literal Search

```javascript
function search(value, query) {
  const source = normalize(value);
  const needle = normalize(query);
  if (!needle) return [];
  const haystack = source.toLocaleLowerCase();
  const target = needle.toLocaleLowerCase();
  const matches = [];
  let start = 0;
  while (start <= haystack.length - target.length) {
    const index = haystack.indexOf(target, start);
    if (index < 0) break;
    matches.push({ index, text: source.slice(index, index + needle.length) });
    start = index + needle.length;
  }
  return matches;
}
```

Limitation: Case Mapping যদি Text Length বদলায়, Original Index ও Lowercased Index ভিন্ন হতে পারে। এই Beginner Tool-এ সাধারণ বাংলা/English Input-এর জন্য পদ্ধতিটি ব্যবহার করা হয়েছে। Full Unicode Search Engine তৈরি করতে Offset Mapping দরকার।

Step 4: Safe Highlight

```javascript
const preview = document.getElementById('preview');
preview.replaceChildren();
let cursor = 0;
for (const match of search(content, query)) {
  preview.append(document.createTextNode(content.slice(cursor, match.index)));
  const mark = document.createElement('mark');
  mark.textContent = match.text;
  preview.append(mark);
  cursor = match.index + match.text.length;
}
preview.append(document.createTextNode(content.slice(cursor)));
```

Full UI-তে Analysis, Copy Slug, Error Message, Responsive Styling এবং Accessibility Label রয়েছে।

Run Instructions

```terminal
# Folder: source-code/chapter-08
# Browser-এ index.html খুলুন; network প্রয়োজন নেই।
node --check text-tools.js
node --check app.js
node text-tools.test.js
```

Node.js Test-এর জন্য Modern Node Version প্রয়োজন। Browser-এ `Intl.Segmenter` না থাকলে Code Point Count Fallback দেখাবে; সেই ফল Grapheme Count-এর সমতুল্য নয়।

৮.১৪ Debugging এবং Common Mistakes

সমস্যা	ভুল ধারণা	সঠিক সমাধান
Emoji Length 2	একটি Emoji সবসময় 1 Code Unit	Code Point/Grapheme পার্থক্য বুঝুন
Search Result -1	`indexOf()` False দেয়	`-1` দিয়ে Not Found পরীক্ষা করুন
`replace()` একটি Match বদলায়	সব Match বদলায়	`replaceAll()` বা Global Regex
`split(',')` দিয়ে CSV	সব CSV Simple	Proper CSV Parser ব্যবহার করুন
`innerHTML` দিয়ে Highlight	User Text সবসময় Safe	Text Node + `textContent`
Slug Duplicate	একই Title মানেই Unique URL	Uniqueness Strategy যোগ করুন
`.toLowerCase()` সব Locale-এ সমান	Universal Case Mapping	Locale-Aware Search Design

৮.১৫ অনুশীলনী — ১০টি Exercise

1. একটি String-এর প্রথম ও শেষ Character বের করুন।
2. `slice()` দিয়ে একটি Filename-এর Extension আলাদা করুন।
3. Case-Insensitive Search Function লিখুন।
4. Multiple Spaces এক Space-এ রূপান্তর করুন।
5. `map()` দিয়ে Array of Names-কে Trim করুন।
6. Unicode Code Point ও UTF-16 Length-এর পার্থক্য দেখান।
7. `slugify()` Function লিখুন।
8. একটি Paragraph-এর Top Three Frequent Words বের করুন।
9. ২৫ Character-এর Excerpt তৈরি করুন।
10. User Search Term Highlight করার সময় XSS এড়ানোর DOM Strategy লিখুন।

৮.১৬ Exercise Solutions

JAVASCRIPT

```
// 1
const text = 'JavaScript';
console.log(text.at(0), text.at(-1));
// 2
const filename = 'report.final.pdf';
console.log(filename.slice(filename.lastIndexOf('.') + 1)); // pdf
// 3
const includesIgnoreCase = (source, query) =>
  String(source).toLocaleLowerCase().includes(String(query).toLocaleLowerCase());
// 4
console.log('A   B  C'.replace(/\s+/g, ' '));
// 5
console.log(['  Amina ', ' Karim  '].map(name => name.trim()));
// 6
console.log('🙂'.length, Array.from('🙂').length); // 2, 1
```

```
// 7
const slug = text => String(text).trim().toLocaleLowerCase().replace(/[^\p{L}\p{M}\p{N}]+/gu,'-
').replace(/^-+|-+$/g,'');
console.log(slug('Learn JavaScript!'));
// 8
const top3 = paragraph => TextTools.frequency(paragraph).slice(0, 3);
// 9
console.log(TextTools.excerpt('JavaScript is a versatile language.', 25));
// 10: document.createTextNode() + mark.textContent; user text কখনো HTML হিসেবে parse নয়
```

Note: Solution 8 ও 9 চালাতে Project-এর `text-tools.js` Load করা থাকতে হবে।

৮.১৭ Knowledge Check — ১০টি MCQ

1. String Primitive কি Mutable? A. Yes B. No C. Only in Browser D. Only Emoji
2. `'🙂'.length` কত? A. 1 B. 2 C. 4 D. 0
3. Negative Index Support করে কোনটি? A. `at()` B. `charAt()` C. `indexOf()` D. `startsWith()`
4. `indexOf()`-এ Not Found Result? A. false B. null C. -1 D. undefined
5. সব Literal Occurrence বদলাতে? A. `replaceAll()` B. `trim()` C. `slice()` D. `split()`
6. Unicode Canonical Composition? A. NFC B. CSV C. DOM D. JSON
7. Template Literal Delimiter? A. `'` B. `"` C. Backtick D. Slash
8. Safe Plain Text Rendering? A. `innerHTML` B. `textContent` C. `eval` D. `document.write`
9. Unicode Letter Match-এর জন্য কোন Regex Class? A. `\p{L}` with `u` B. `\d` only C. `.` only D. `\s`
10. Search-Friendly Slug-এ কোনটি প্রয়োজন হতে পারে? A. Uniqueness Check B. `eval()` C. Password D. Global Variable

Answer Key ও Explanation

Q	Answer	কারণ
1	B	String Primitive Immutable
2	B	Emoji UTF-16-এ দুটি Code Unit
3	A	`at(-1)` শেষ Element দেয়
4	C	`indexOf()` Not Found হলে -1
5	A	`replaceAll()` সব Occurrence বদলায়
6	A	NFC Canonical Composition করে
7	C	Template Literal Backtick দিয়ে লেখা হয়
8	B	`textContent` HTML Parse করে না
9	A	Unicode Property Escape ও `u` Flag
10	A	Duplicate Slug URL Collision তৈরি করতে পারে

৮.১৮ Final Project Challenge

Project-এ তিনটি Feature যোগ করুন: (১) Local Draft Save/Load, (২) Stopword Exclusion Option, (৩) Keyword Density Percentage। প্রতিটি Feature-এর জন্য অন্তত একটি Automated Test লিখুন। Stopword List ভাষাভেদে ভিন্ন হবে—বাংলা ও English-এর জন্য আলাদা Config ব্যবহার করুন।

৮.১৯ Chapter Summary ও Next Step

String Processing শুধু `slice()` বা `replace()` শেখার বিষয় নয়; Unicode, Locale, Security, Search Quality এবং Content Usability-ও গুরুত্বপূর্ণ। এই অধ্যায়ের Search-Friendly Content Tool একটি Testable, Browser-Based Example। পরবর্তী **Chapter 9 — DOM Manipulation, Events এবং Interactive Web Applications**-এ Event Handling, Dynamic UI ও Task Manager Project তৈরি করব।